

ROAD TO HALL OF FAME - ACTIVIDAD 10
FILE PATH TRAVERSAL, TRAVERSAL SEQUENCES
STRIPPED WITH SUPERFLUOUS URL-DECODE

Mtro. Servando López Contreras

Informe realizado por:

181583 Angel Aaron Acuña Quezada

Contenido

Datos del Laboratorio	3
Marco Teórico y Fundamentación Técnica	3
Procedimiento	4
Explicación del Payload	7
Mitigación Recomendada.....	8
Referencias	8

Datos del Laboratorio

- **Nombre del laboratorio:** *File path traversal, traversal sequences stripped with superfluous URL-decode*
- **Nivel:** Practitioner
- **Tipo de Explotación:** File Path Traversal
- **Objetivo del laboratorio:** Explorar la vulnerabilidad de File Path Traversal a través del método de petición "GET" del protocolo HTTP, con el fin de poder leer el archivo `/etc/passwd`.

Marco Teórico y Fundamentación Técnica

Concepto: De acuerdo con OWASP, un ataque de *Path Traversal* (también conocido como *Directory Traversal*) tiene como objetivo acceder a archivos y directorios que están almacenados fuera de la carpeta raíz web. Al manipular las variables que hacen referencia a los archivos usando secuencias de "punto-punto-barra" (`../`) o rutas absolutas permite a un atacante acceder a archivos y directorios en el sistema de archivos, incluyendo código fuente de la aplicación, configuraciones y archivos críticos del sistema. En el caso particular de este laboratorio, la aplicación primero verifica que la entrada no contenga secuencias de traversal en texto plano (`../`) y la rechaza si las encuentra. Sin embargo, después de pasar esa validación, la aplicación realiza una decodificación URL adicional ('superflua') sobre la entrada ya validada. Esto permite enviar una secuencia doblemente codificada en URL que, tras el primer decode automático del servidor, no contiene aún `../` literal (por lo que pasa el filtro), pero que al ser decodificada una segunda vez por la aplicación revela la secuencia de traversal real.

Impacto en la Triada CIA: Esta vulnerabilidad compromete de manera directa la "Confidencialidad" de la información, ya que expone datos sensibles del sistema operativo.

Perspectiva OWASP: Clasificada dentro de la categoría de *Broken Access Control* (Control de Acceso Roto), ocurre cuando la aplicación utiliza la entrada del usuario para construir una ruta de archivo sin validar o sanear adecuadamente dicha entrada, permitiendo escapar de la estructura de directorios autorizada.

Procedimiento

File path traversal, traversal sequences stripped with superfluous URL-decode

Back to lab description »

LAB Not solved

Home

WE LIKE TO SHOP

Se ingresó a la tienda virtual del laboratorio, en la cual podemos ver que carga varias imágenes de productos.

Intercept HTTP history WebSockets history Match and replace Proxy settings

Filter settings: Hiding specific extensions

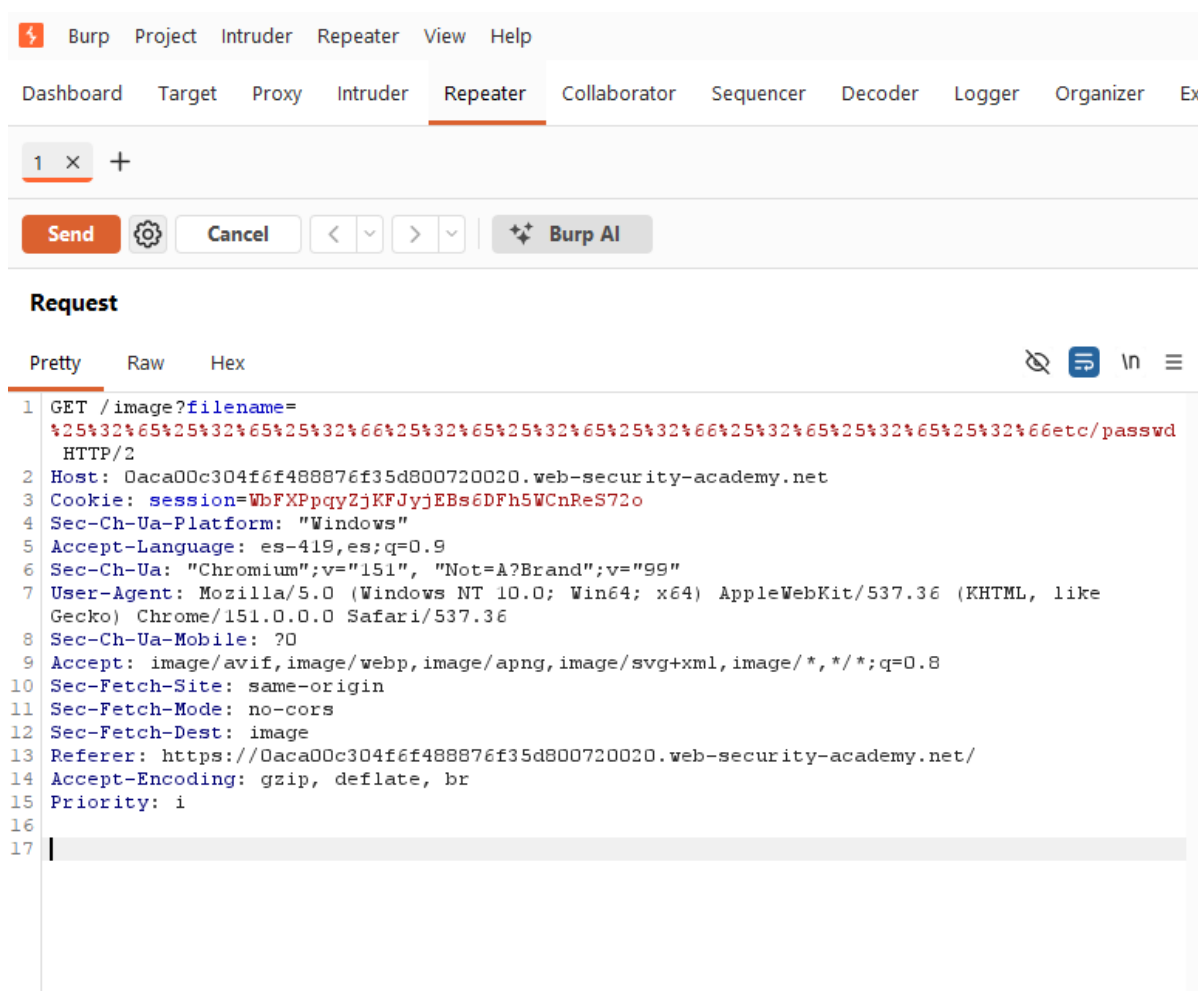
#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	Listener port	Start response
1	https://portswigger.net	GET	/academy/labs/launch/off4cb891d7f7d...		✓	302	2426					✓	13.226.204.28	SessionId=CFD.RP+P...	18:23:15 26 Se...	6000	634
2	https://portswigger.net	GET	/users?returnurl=%2Facademy%2Flabs...		✓	302	3627					✓	13.226.204.28	SessionId=CFD.RP+P...	18:23:16 26 Se...	6000	629
3	https://login.portswigger.net	GET	/authorize?client_id=1F1NGMosqueuH...		✓	302	2453	HTML				✓	104.18.34.171	did=v%3AvO%3A6c...	18:23:17 26 Se...	6000	272
4	https://login.portswigger.net	GET	/u/login?state=hKfQ25A1TEZHHh3bip...		✓	200	38727	HTML		Log in portswigger.net		✓	104.18.34.171		18:23:18 26 Se...	6000	252
9	https://daca00c304f848887f935...	GET	/		✓	200	10006	HTML		File path traversal, traver...		✓	34.246.129.62	session=wbFXpgay...	18:23:48 26 Se...	6000	206
15	https://daca00c304f848887f935...	GET	/image?filename=65.jpg		✓	200	156253	JPEG				✓	34.246.129.62		18:23:50 26 Se...	6000	843
17	https://daca00c304f848887f935...	GET	/image?filename=55.jpg		✓	200	175583	JPEG				✓	34.246.129.62		18:23:50 26 Se...	6000	253
18	https://daca00c304f848887f935...	GET	/image?filename=21.jpg		✓	200	194766	JPEG				✓	34.246.129.62		18:23:50 26 Se...	6000	1009
19	https://daca00c304f848887f935...	GET	/image?filename=30.jpg		✓	200	446412	JPEG				✓	34.246.129.62		18:23:50 26 Se...	6000	1255
21	https://daca00c304f848887f935...	GET	/image?filename=41.jpg		✓	200	185923	JPEG				✓	34.246.129.62		18:23:50 26 Se...	6000	1180
22	https://daca00c304f848887f935...	GET	/image?filename=62.jpg		✓	200	174829	JPEG				✓	34.246.129.62		18:23:50 26 Se...	6000	1167
24	https://daca00c304f848887f935...	GET	/image?filename=38.jpg		✓	200	277107	JPEG				✓	34.246.129.62		18:23:50 26 Se...	6000	1420
25	https://daca00c304f848887f935...	GET	/image?filename=28.jpg		✓	200	127703	JPEG				✓	34.246.129.62		18:23:50 26 Se...	6000	1408
27	https://daca00c304f848887f935...	GET	/image?filename=16.jpg		✓	200	133514	JPEG				✓	34.246.129.62		18:23:50 26 Se...	6000	1219
28	https://daca00c304f848887f935...	GET	/image?filename=61.jpg		✓	200	271604	JPEG				✓	34.246.129.62		18:23:50 26 Se...	6000	1372
29	https://daca00c304f848887f935...	GET	/image?filename=53.jpg		✓	200	301401	JPEG				✓	34.246.129.62		18:23:50 26 Se...	6000	1240
31	https://daca00c304f848887f935...	GET	/image?filename=40.jpg		✓	200	159107	JPEG				✓	34.246.129.62		18:23:50 26 Se...	6000	1414
32	https://daca00c304f848887f935...	GET	/image?filename=31.jpg		✓	200	127667	JPEG				✓	34.246.129.62		18:23:50 26 Se...	6000	1404
33	https://daca00c304f848887f935...	GET	/image?filename=65.jpg		✓	200	155467	JPEG				✓	34.246.129.62		18:23:50 26 Se...	6000	1450
34	https://daca00c304f848887f935...	GET	/image?filename=34.jpg		✓	200	170791	JPEG				✓	34.246.129.62		18:23:50 26 Se...	6000	1452
36	https://daca00c304f848887f935...	GET	/image?filename=11.jpg		✓	200	160826	JPEG				✓	34.246.129.62		18:23:50 26 Se...	6000	1446
37	https://daca00c304f848887f935...	GET	/image?filename=72.jpg		✓	200	212277	JPEG				✓	34.246.129.62		18:23:50 26 Se...	6000	1463
38	https://daca00c304f848887f935...	GET	/image?filename=27.jpg		✓	200	201043	JPEG				✓	34.246.129.62		18:23:50 26 Se...	6000	1426
39	https://daca00c304f848887f935...	GET	/image?filename=7.jpg		✓	200	194100	JPEG				✓	34.246.129.62		18:23:50 26 Se...	6000	1476
40	https://daca00c304f848887f935...	GET	/image?filename=5.jpg		✓	200	184975	JPEG				✓	34.246.129.62		18:23:50 26 Se...	6000	1490
41	https://daca00c304f848887f935...	GET	/academyLabHeader		✓	101	147					✓	34.246.129.62		18:23:50 26 Se...	6000	197
43	https://daca00c304f848887f935...	GET	/product?productId=1		✓	200	4376	HTML		File path traversal, traver...		✓	34.246.129.62		18:25:08 26 Se...	6000	221
48	https://daca00c304f848887f935...	GET	/image?filename=65.jpg		✓	200	156253	JPEG				✓	34.246.129.62		18:25:10 26 Se...	6000	216
51	https://daca00c304f848887f935...	GET	/academyLabHeader		✓	101	147					✓	34.246.129.62		18:25:10 26 Se...	6000	209
52	https://daca00c304f848887f935...	GET	/product?productId=2		✓	200	3808	HTML		File path traversal, traver...		✓	34.246.129.62		18:25:24 26 Se...	6000	204
54	https://daca00c304f848887f935...	GET	/image?filename=30.jpg		✓	200	446412	JPEG				✓	34.246.129.62		18:25:30 26 Se...	6000	251
55	https://daca00c304f848887f935...	GET	/academyLabHeader		✓	101	147					✓	34.246.129.62		18:25:30 26 Se...	6000	197
56	https://daca00c304f848887f935...	GET	/product?productId=2		✓	200	3808	HTML		File path traversal, traver...		✓	34.246.129.62		18:25:35 26 Se...	6000	193
57	https://daca00c304f848887f935...	GET	/image?filename=30.jpg		✓	200	446412	JPEG				✓	34.246.129.62		18:25:37 26 Se...	6000	227
58	https://daca00c304f848887f935...	GET	/academyLabHeader		✓	400	130	JSON				✓	34.246.129.62		18:25:37 26 Se...	6000	195

The screenshot displays the Burp Suite interface. The 'HTTP history' tab is active, showing a list of intercepted requests. A 'Filter settings' dialog is open, allowing the user to refine the displayed requests. The dialog includes several filter categories:

- Filter by request type:**
 - ☐ Show only in-scope items
 - ☐ Hide items without responses
 - ☐ Show only parameterized requests
- Filter by MIME type:**
 - ☒ HTML
 - ☒ Script
 - ☒ XML
 - ☐ CSS
 - ☒ Other text
 - ☒ Images
 - ☒ Flash
 - ☒ Other binary
- Filter by status code:**
 - ☒ 2xx [success]
 - ☒ 3xx [redirection]
 - ☒ 4xx [request error]
 - ☒ 5xx [server error]
- Filter by search term:**
 - ☐ Regex
 - ☐ Case sensitive
 - ☐ Negative search
- Filter by file extension:**
 - ☐ Show only: asp,aspx,jsp,php
 - ☒ Hide: png,ico,css,woff,woff2,tif,svg
- Filter by annotation:**
 - ☐ Show only items with notes
 - ☐ Show only highlighted items
- Filter by listener:**
 - Port:

The main window shows a list of requests. The first request is a GET to /image with a status code of 200. The response is shown in the 'Response' tab, indicating a successful 200 OK status with content type text/html.

Durante la fase de reconocimiento en Burp Suite, se identificó un patrón inseguro en las peticiones GET hacia /image. La aplicación emplea el parámetro filename para la carga de recursos. Este parámetro se convierte en un vector ideal para inyectar salto de directorio. Para poder visualizar estas peticiones en el HTTP history, fue necesario ajustar el filtro y habilitar la casilla 'Images' dentro de 'Filter by MIME type', ya que por defecto estaba desmarcada y ocultaba las peticiones hacia recursos de tipo imagen, incluyendo las que utilizan el parámetro filename.



Preparamos el payload codificando doblemente en URL la secuencia de traversal: `%25%32%65%25%32%65%25%32%66` repetido tres veces seguido de `etc/passwd`. Al decodificarse, `%25` se convierte en `%`, por lo que la cadena completa se traduce, tras un primer decode, en `%2e%2e%2f%2e%2e%2f%2e%2e%2fetc/passwd` — es decir, aún no contiene `../` literal y logra pasar el filtro de la aplicación. Sin embargo, al aplicar la aplicación su decodificación adicional (superflua), esta cadena se decodifica una segunda vez, revelando finalmente `../../etc/passwd`.

El parámetro filename indica al servidor qué archivo leer del disco duro. En este laboratorio, la aplicación primero valida que la entrada no contenga secuencias de traversal en texto plano, rechazando cualquier ../ literal. El problema surge porque, después de esa validación, la aplicación decodifica la entrada una vez más de manera innecesaria ('superflua'). Al enviar la secuencia doblemente codificada en URL, el primer decode (realizado automáticamente al procesar la petición) no produce todavía ../, por lo que la validación se cumple sin detectar nada sospechoso. Sin embargo, el segundo decode que realiza la aplicación sí traduce la cadena en la secuencia de traversal real, permitiendo ascender en la jerarquía de directorios y llegar hasta /etc/passwd..

Mitigación Recomendada

- **Evitar entradas directas:** Lo más efectivo es evitar, lo más que se pueda, pasar la entrada directa al sistema de archivos. En su lugar, se podrían usar identificadores en una base de datos para localizar un archivo.
- **Validar después de decodificar, no antes:** el filtro de secuencias de traversal debe aplicarse después de cualquier decodificación, y nunca antes. Realizar validaciones sobre datos aún codificados permite que un atacante oculte el patrón malicioso tras una o varias capas de codificación que se revelan más adelante en el procesamiento.
- **Evitar decodificaciones múltiples o innecesarias:** la aplicación no debería decodificar una entrada más veces de las estrictamente necesarias. Cada decode adicional es una oportunidad para que contenido previamente "limpio" se convierta en peligroso.
- **Rutas Absolutas y canonicalización:** Tras recibir la entrada, la aplicación debe anexarla al directorio base estricto y utilizar funciones de canonicalización (ej. `realpath()` o `os.path.abspath()`), verificando que la ruta resultante comience exactamente con el directorio autorizado.

Referencias

OWASP. (s.f.). *Path Traversal*. OWASP Community. Recuperado el 26 de septiembre de 2026, de https://community.owasp.org/attacks/Path_Traversal